On the Information Bottleneck Theory of Deep Learning

A Reproduction and Critical Study

after Saxe, Bansal, Dapello, Advani, Kolchinsky, Tracey & Cox
***J. Stat. Mech.* (2019) 124020**

Course project — Information Theory and Inference

July 29, 2026

Abstract

The information bottleneck (IB) theory of deep learning makes three claims: that training passes through a fitting phase followed by a distinct *compression* phase; that compression causes the good generalisation of deep networks; and that compression is driven by the diffusion-like noise of stochastic gradient descent. This report reproduces, from scratch, the analysis of Saxe et al. [2019], which shows that none of the three survives scrutiny. We implement four mutual information estimators, validate them against analytic values and against the original authors' code to a relative accuracy of 10^{-10}, and re-derive the paper's results on its own datasets. Our replication confirms every substantive conclusion. A `tanh` network compresses by 10.7 bits; changing one identifier to `relu` reduces this to 0.5 bits while *improving* test accuracy. Holding the network entirely fixed and changing only the bin edges reduces the deepest layer's compression from 5.59 to 0.32 bits. In deep linear networks, where the mutual information is available in closed form and no estimator is involved, no compression occurs at all — and two networks with indistinguishable information planes differ in generalisation error by a factor of twenty. Full-batch gradient descent, which contains no stochasticity whatsoever, compresses *more* than SGD (17.9 versus 10.7 bits). We also report three findings that go beyond the paper: a quantification of *when* task-irrelevant compression begins (at 39 % of fitting, hence interleaved rather than sequential); a demonstration that the Kraskov estimator is invalid for ReLU layers, whose atom at zero makes 45 % of activation vectors identical; and an extension to a small transformer, where we test whether softmax attention and LayerNorm — both bounded operations the paper never examined — reintroduce artefactual compression into an architecture built from non-saturating nonlinearities.

Contents

1 Introduction

Deep networks work, and it is not settled why. One influential proposal analyses learning through the lens of information theory [Tishby and Zaslavsky, 2015, Shwartz-Ziv and Tishby, 2017]. On this view each hidden layer T is a summary statistic of the input X, and learning is the search for a representation that discards as much of X as possible while retaining what predicts the target Y. The natural picture is the *information plane*: a plot of $\mathrm{I}(T;Y)$ against $\mathrm{I}(X;T)$, one curve per layer, parameterised by training time.

The appeal is real. Mutual information is architecture-agnostic, so different models become comparable in a common currency; and the IB principle [Tishby et al., 1999] supplies an optimal frontier against which any representation can be judged.

Shwartz-Ziv and Tishby [2017] reported a striking empirical finding in this plane: training appears to have two phases — an initial *fitting* phase in which $\mathrm{I}(X;T)$ and $\mathrm{I}(T;Y)$ both rise, and a subsequent *compression* phase in which $\mathrm{I}(X;T)$ falls while $\mathrm{I}(T;Y)$ does not. Three claims were built on it.

Claim 1 (Two phases)**.** *Deep network training consists of a fitting phase followed by a distinct compression phase.*

Claim 2 (Causality)**.** *The compression phase is causally responsible for good generalisation.*

Claim 3 (Mechanism)**.** *Compression arises from the diffusion-like behaviour of SGD.*

Saxe et al. [2019] argue that none of the three holds in general, and that all three reflect assumptions made in order to compute a finite mutual information at all. This report reproduces their argument from first principles.

What this report contributes. Everything is re-implemented from the paper's equations: the networks, the four estimators, the training instrumentation. We validate each estimator against analytically known values and, where a reference exists, against the original authors' code. Beyond replication we (i) quantify the timing of task-irrelevant compression, which the paper asserts but does not measure; (ii) identify and diagnose a failure of the Kraskov estimator on ReLU layers that the paper's appendix does not flag; (iii) document two internal inconsistencies in the paper's own specification; and (iv) extend the analysis to a transformer.

Deliverables. Six Jupyter notebooks (`01`–`06`) reproduce the paper section by section, plus a seventh (`07`) for the transformer extension. They rest on a documented library, `ibdl/`, with a validation suite of 46 checks.

2 Information-theoretic background

2.1 The information bottleneck principle

Given jointly distributed (X, Y), the IB problem [Tishby et al., 1999] seeks a stochastic map $p(t \mid x)$ minimising

$$\mathcal{L}_{\mathrm{IB}} = \mathrm{I}(X;T) - \beta\, \mathrm{I}(T;Y), \tag{1}$$

trading compression of the input against prediction of the target. Sweeping β traces an optimal frontier in the information plane. For jointly Gaussian variables the solution is known in closed form [Chechik et al., 2005].

Two structural facts underpin the IB reading of a deep network. First, a feed-forward network forms a Markov chain

$$Y \to X \to T_1 \to T_2 \to \cdots \to \hat{Y}, \tag{2}$$

so by the *data processing inequality* (DPI) [Cover and Thomas, 2006]

$$\mathrm{I}(X;T_1) \geq \mathrm{I}(X;T_2) \geq \cdots, \qquad \mathrm{I}(T_1;Y) \geq \mathrm{I}(T_2;Y) \geq \cdots. \tag{3}$$

Layers should therefore appear in order along both axes. Second, mutual information is invariant under invertible reparametrisation, so two networks that differ only by a change of internal coordinates should be indistinguishable.

Both facts will turn out to fail for the quantity actually plotted — not because the theorems are wrong, but because the quantity plotted is not the one they apply to.

2.2 The obstruction: deterministic networks carry infinite information

This is the technical heart of the matter. Let a hidden layer compute $h = f(X)$ with f continuous and deterministic. Then

$$\mathrm{I}(h;X) = \mathrm{H}(h) - \mathrm{H}(h \mid X). \tag{4}$$

Given X, the activity h is a point mass at $f(X)$; the differential entropy of a point mass is $-\infty$; hence for any layer with finite $\mathrm{H}(h)$,

$$\boxed{\mathrm{I}(h;X) = \infty} \tag{5}$$

at *every* point in training [Laughlin, 1981, Tishby and Zaslavsky, 2015]. A constant cannot have a fitting phase or a compression phase.

Every finite number ever plotted in an information plane is therefore produced by an additional assumption, of which two are standard:

(i) **Binning.** Discretise h into $T = \mathrm{bin}(h)$. Then T is a deterministic function of X but discrete, so $\mathrm{H}(T \mid X) = 0$ and $\mathrm{I}(T;X) = \mathrm{H}(T)$ is finite.

(ii) **Added noise.** Define $T = h + Z$ with $Z \perp X$. Then $\mathrm{H}(T \mid X) = \mathrm{H}(Z)$ is finite, so $\mathrm{I}(T;X) = \mathrm{H}(T) - \mathrm{H}(Z)$ is finite.

Neither is part of the network. Both are choices made by the analyst, and *the trajectory reports on the choice as much as on the network.* This is the thesis of the paper and the organising idea of this report.

Two consequences follow immediately and are worth stating, because they undercut the two structural facts above.

The DPI does not apply. Noise added post hoc at each layer does not propagate. For $X \to h_1 \to h_2$ with analysis variables $T_i = h_i + Z_i$, the chain is $X \to h_1 \to h_2 \to T_2$, not $X \to h_1 \to T_1 \to T_2$. The DPI therefore bounds $\mathrm{I}(X;h_1) \geq \mathrm{I}(X;T_2)$ and says nothing about $\mathrm{I}(X;T_1)$ versus $\mathrm{I}(X;T_2)$. Apparent DPI violations in published information planes are artefacts of this.

Invariance is destroyed. $T = h + Z$ is not invariant under reparametrisation of h, because the noise enters at a fixed scale. We give a concrete demonstration in §5.2.

2.3 The four estimators

We implement all four routes to a finite number used in the paper. Each is defined below and validated in §3.3.

(1) Binning. With the input alphabet finite and fully enumerated, and taken uniform,

$$\mathrm{I}(T;X) = \mathrm{H}(T) - \underbrace{\mathrm{H}(T \mid X)}_{=0} = -\sum_i p_i \log p_i, \tag{6}$$

where p_i is the probability that an input lands in bin i. For a monotone nonlinearity and scalar Gaussian input this is exact via the Gaussian CDF,

$$p_i = \Phi\big(f^{-1}(b_{i+1})/w_1\big) - \Phi\big(f^{-1}(b_i)/w_1\big). \tag{7}$$

The label Y is genuinely discrete, so $\mathrm{I}(T;Y) = \mathrm{H}(T) - \sum_y p(y)\mathrm{H}(T \mid Y{=}y)$ needs no further assumption.

(2) Kernel density bounds. Assume $T = h + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$, and take the empirical input distribution. Then $p(T)$ is *exactly* a uniform mixture of P Gaussians centred at the h_i, and pairwise distances bound its entropy [Kolchinsky and Tracey, 2017]. Since $\mathrm{H}(T \mid X) = \mathrm{H}(\epsilon)$,

$$\mathrm{I}(T;X) \;\le\; -\frac{1}{P}\sum_i \log \frac{1}{P}\sum_j \exp\!\left(-\frac{1}{2}\frac{\|h_i - h_j\|_2^2}{\sigma^2}\right), \tag{8}$$

with the Bhattacharyya lower bound obtained by replacing $\sigma^2 \to 4\sigma^2$ inside the exponent. The normalising constants cancel exactly — a useful check on the algebra.

(3) Kraskov k-NN. If $T = h + Z$ with $Z \perp X$ then $\mathrm{I}(T;X) = \mathrm{H}(T) - \mathrm{H}(Z) = \mathrm{H}(T) - c$ for a *constant* c, so compression can be read off $\mathrm{H}(T)$ alone without fixing a noise level. The Kozachenko–Leonenko estimator [Kozachenko and Leonenko, 1987, Kraskov et al., 2004] gives

$$\hat{\mathrm{H}} = \frac{d}{P}\sum_{i=1}^{P} \log(r_i + \varepsilon) + \frac{d}{2}\log \pi - \log \Gamma(d/2 + 1) + \psi(P) - \psi(k), \tag{9}$$

with r_i the distance to the k-th nearest neighbour. We show in §4.4 that this estimator is *invalid* for ReLU layers.

(4) Exact linear–Gaussian. For a deep linear network with $X \sim \mathcal{N}(0, \Sigma_X)$, a hidden layer applying the cumulative map $\bar{W} = W_l \cdots W_1$, and analysis noise σ_{MI},

$$\mathrm{I}(T;X) = \tfrac{1}{2}\log\big|\bar{W}\Sigma_X\bar{W}^\top + \sigma_{MI}^2 I\big| - \tfrac{1}{2}\log\big|\sigma_{MI}^2 I\big|, \tag{10}$$

$$\mathrm{I}(T;Y) = \mathrm{H}(T) + \mathrm{H}(Y) - \mathrm{H}(T,Y), \tag{11}$$

all in closed form. This estimator has *no error*, which makes it decisive.

Subspace information. For §7 we need the information a layer carries about a *subset* S of input coordinates. Conditioning on X_S leaves the complement acting as noise:

$$\mathrm{I}(T;X_S) = \tfrac{1}{2}\log\big|\bar{W}\Sigma_X\bar{W}^\top + \sigma_{MI}^2 I\big| - \tfrac{1}{2}\log\big|\bar{W}_{\bar{S}}\Sigma_{\bar{S}}\bar{W}_{\bar{S}}^\top + \sigma_{MI}^2 I\big|. \tag{12}$$

Since $X_{\mathrm{rel}} \perp X_{\mathrm{irrel}}$, the chain rule gives

$$\mathrm{I}(X;T) = \mathrm{I}(X_{\mathrm{rel}};T) + \mathrm{I}(X_{\mathrm{irrel}};T) + \underbrace{\mathrm{I}(X_{\mathrm{rel}};X_{\mathrm{irrel}} \mid T)}_{\ge 0}, \tag{13}$$

so the parts are *sub-additive*: observing T makes two a priori independent blocks dependent ("explaining away"). We use this in §7 to explain how $\mathrm{I}(X_{\mathrm{irrel}};T)$ can fall while $\mathrm{I}(X;T)$ rises.

3 Methods and validation

3.1 Datasets and architectures

The 12-bit dataset. The synthetic dataset of Shwartz-Ziv and Tishby [2017]: all $2^{12} = 4096$ twelve-bit patterns, each appearing exactly once, with a binary label ($P(Y{=}1) = 0.517$, so $\mathrm{H}(Y) = 0.9992$ bits). We use the authors' own `var_u.mat`. The crucial property is that the input alphabet is *finite and fully enumerated*: taking it uniform gives $\mathrm{H}(X) = \log_2 4096 = 12$ bits exactly, and binning-based mutual information evaluated over all 4096 patterns has *no sampling error*. The network is 12–10–7–5–4–3–2, the last layer two sigmoidal units, trained by SGD on 80 % of the patterns with 256 samples per batch.

Linear student–teacher. $X \sim \mathcal{N}(0, I/N_i)$, $Y = W_o X + \epsilon_o$ with $W_{o,ij} \sim \mathcal{N}(0, \sigma_w^2)$ and $\epsilon_o \sim \mathcal{N}(0, \sigma_o^2)$; $\mathrm{SNR} = \sigma_w^2/\sigma_o^2$. The student is a deep linear network. The exact generalisation error is

$$E_g = \mathrm{tr}\big[(W_o - W_{\mathrm{tot}})\Sigma_X (W_o - W_{\mathrm{tot}})^\top\big] + \sigma_o^2 N_o. \tag{14}$$

MNIST. 784–1024–20–20–20–10, SGD with minibatches of 128. The three narrow layers are the paper's choice, made so the kernel density estimator stays well conditioned.

3.2 Two inconsistencies in the paper's specification

Reproducing the paper exactly required resolving two internal conflicts. We record them because they affect any attempt to reproduce this work.

(i) Missing Σ_X and a missing factor of $\frac{1}{2}$. The text states $X \sim \mathcal{N}(0, \frac{1}{N_i} I_{N_i})$, but equations (5), (6) and (G.1)–(G.3) are written with Σ_X suppressed, as though $\Sigma_X = I$; and equation (6) additionally drops the factor $\frac{1}{2}$ that appears correctly in (G.2). We carry both explicitly, as in (10) and (14). This is a constant offset and a scale on the axes, not a change to any conclusion; and it places $\mathrm{I}(X;T)$ in the range 0.5–2.5 bits, matching the axes of the paper's own figure 3D.

(ii) Figures 3 and 4 are described identically but conclude oppositely. Figure 3 ("generalises well, minimal overtraining") and figure 4 ("overfitting is substantial") are both specified with $N_i = P = 100$ and one hidden layer of 100 units. Since Advani and Saxe [2017] — cited by the paper for exactly this point — show overtraining is *worst* at $P = N_i$, the two cannot differ in the stated parameters. We determined empirically that the distinguishing factor is the optimiser and the training horizon: figure 3 is batch gradient descent over 500 epochs (its colour bar tops out at epoch 499), figure 4 is SGD with 5 samples per batch, i.e. 20 updates per epoch. With that reading both regimes reproduce cleanly.

3.3 Estimator validation

Because the entire argument rests on the estimators, we validate them before using them. Table 1 summarises; all 46 checks pass.

3.4 Instrumentation

Per-sample gradients for the SNR statistics of equations (I.1)–(I.2),

$$m_l = \big\| \langle \partial E/\partial W_l \rangle \big\|_F, \qquad s_l = \big\| \mathrm{STD}\,(\partial E/\partial W_l) \big\|_F, \qquad \mathrm{SNR}_l = m_l/s_l, \tag{15}$$

Estimator	Check	Result
Binning	P distinct patterns $\Rightarrow \mathrm{I} = \log_2 P$	exact to 10^{-12}
Binning	$T = Y \Rightarrow \mathrm{I}(T;Y) = \mathrm{H}(Y)$	exact to 10^{-12}
Binning	vs. authors' `simplebinmi.py`	10^{-10} relative
KDE	vs. authors' `kde.py`	10^{-10} relative
KDE	bounds bracket Gaussian channel value	$3.69 \le 3.99 \le 4.69$
KDE	chunked vs. unchunked evaluation	identical (0.0)
Kraskov	$\mathrm{H}(U[0,1]^d) = 0$, $d = 1, 2, 3$	$\lvert\hat{\mathrm{H}}\rvert < 0.03$ nats
Kraskov	$\mathrm{H}(\mathcal{N}(0, \sigma^2 I_2))$	within 0.008 nats
Kraskov	$\mathrm{H}(cX) = \mathrm{H}(X) + d \log c$	exact to 10^{-6}
Exact	scalar channel $\frac{1}{2}\log_2(1 + w^2/\sigma^2_{MI})$	exact to 10^{-12}
Exact	parallel channels add	exact to 10^{-12}
Exact	DPI holds when noise is propagated	confirmed
Exact	sub-additivity (13), synergy ≥ 0	confirmed
Cross	exact value lies within KDE bounds	$1.38 \le 2.08 \le 3.45$
Cross	Kraskov $\hat{\mathrm{H}}(T)$ vs. exact Gaussian $\mathrm{H}(T)$	within 0.01 bits
Trainers	NumPy linear vs. PyTorch autograd	10^{-7}, 17× faster
Trainers	`vmap` per-sample grads vs. loop	10^{-9}

Table 1: Validation of the estimators and trainers. Agreement with the original authors' implementations is to relative 10^{-10}; agreement with analytically known values is to the precision of the estimator.

are computed with `torch.func.vmap(grad(...))` in a single vectorised pass, accumulated in Welford form so memory stays $O(\text{params})$ rather than $O(P \cdot \text{params})$. For MNIST, where per-sample gradients over 825 390 parameters do not fit, we use the paper's alternative protocol and take statistics across SGD minibatches.

4 Compression is a property of the nonlinearity

4.1 An exactly solvable model

Consider the minimal network of the paper's figure 2: a scalar Gaussian input, a single weight, a nonlinearity, and binning,

$$X \sim \mathcal{N}(0,1), \qquad h = f(w_1 X), \qquad T = \mathrm{bin}(h). \tag{16}$$

Equations (6)–(7) give $\mathrm{I}(T;X)$ *exactly*: no sampling, no estimator. Figure 1 shows the result.

The asymptotic slopes are themselves informative: the linear unit gains $\log_2 10 = 3.32$ bits per decade of w_1, and ReLU gains exactly half of that, 1.66 bits, because half of its input distribution lands in the zero bin and only the positive half spreads out. Our measurements reproduce both to three decimals.

Since networks are initialised with small weights and *must* grow them to compute anything nonlinear (a `tanh` network with tiny weights is confined to its linear regime), a `tanh` network traverses the left panel from left to right during training. *That traversal is the reported compression phase.*

4.2 The full network

Figure 2 shows the information plane for the 12–10–7–5–4–3–2 network. The `tanh` result replicates Shwartz-Ziv and Tishby [2017]; the ReLU result does not compress.

The two networks reach comparable accuracy (96.2 % versus 97.3 % test), so the difference is not a difference in learning success. The mechanism is visible directly in the activations: in the

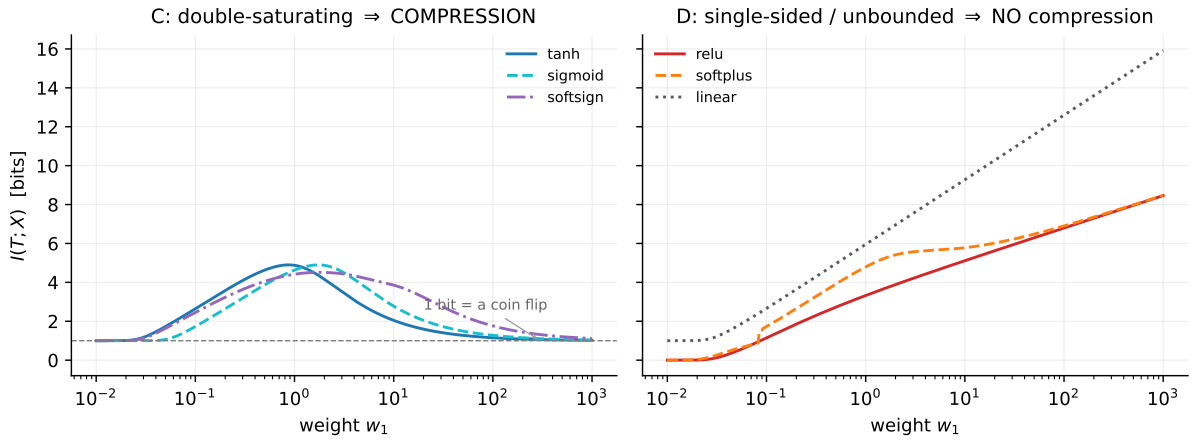


Figure 1: Exact I$(T;X)$ versus weight magnitude in the minimal model. **Left:** double-saturating nonlinearities rise, peak, and fall back to exactly 1 bit — for large w_1 almost every input saturates into one of the two extreme bins, leaving only the sign of the input, a single coin flip. **Right:** single-sided and unbounded nonlinearities rise without bound. Both panels use identical machinery; only the shape of f differs.

	compression per layer (bits lost from peak I$(X;T)$)						
network	L1	L2	L3	L4	L5	L6 (out)	**total**
`tanh`, SGD	0.00	0.02	0.23	2.27	**5.59**	2.56	**10.66**
`tanh`, BGD	0.01	1.34	3.30	3.86	**6.52**	2.84	**17.87**
ReLU, SGD	0.00	0.00	0.00	0.02	0.08	0.37	0.47
ReLU, BGD	0.00	0.00	0.00	0.03	0.11	0.00	0.13

Table 2: Compression by activation function and optimiser. The `tanh` network compresses more than twenty times as much as the ReLU network — and compresses *more* under fully deterministic batch gradient descent than under SGD, contradicting Claim 3.

`tanh` network the fraction of layer-5 units in saturation ($|h| > 0.98$) rises from 0.0 % to 75.4 % over training, while ReLU activations disperse without bound (maximum activity 56.3).

4.3 The measurement, not the network

If compression were a property of the network, it would not move when we change the analysis. It moves. Holding the trained network, its weights and its entire training history *completely fixed*, and changing only the bin edges from evenly spaced in activation to evenly spaced in net input, $b_i \in \tanh(\text{linspace}(-50, 50, N))$, gives Figure 3: layer 5's compression falls from 5.59 to 0.32 bits.

Resolution matters as much as placement. At full machine precision every one of the 4096 patterns receives its own code and I$(X;T)$ is pinned at $\log_2 4096 = 12$ bits with essentially no dynamics — and a real network on real hardware is nearer that regime ($\sim 2^{32}$ levels per unit) than the 30 bins used here.

4.4 Robustness across estimators, and a failure

The KDE bounds confirm the split and refine it (Table 3): `softsign` is double-saturating but approaches its asymptotes more gently than `tanh`, and compresses correspondingly less; `softplus` is a smoothed ReLU and behaves like one. The ordering tracks saturation, not anything about the optimiser.

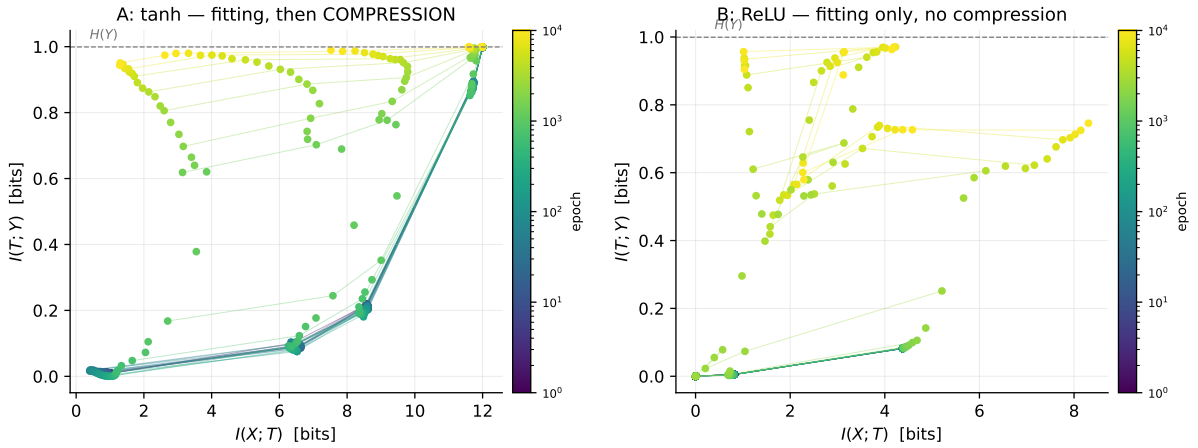


Figure 2: Information plane, binning estimator, all 4096 patterns. **A:** `tanh` — fitting followed by compression, replicating the original result. **B:** ReLU — fitting only. The only ReLU curve that compresses is the output layer, which is still sigmoidal.

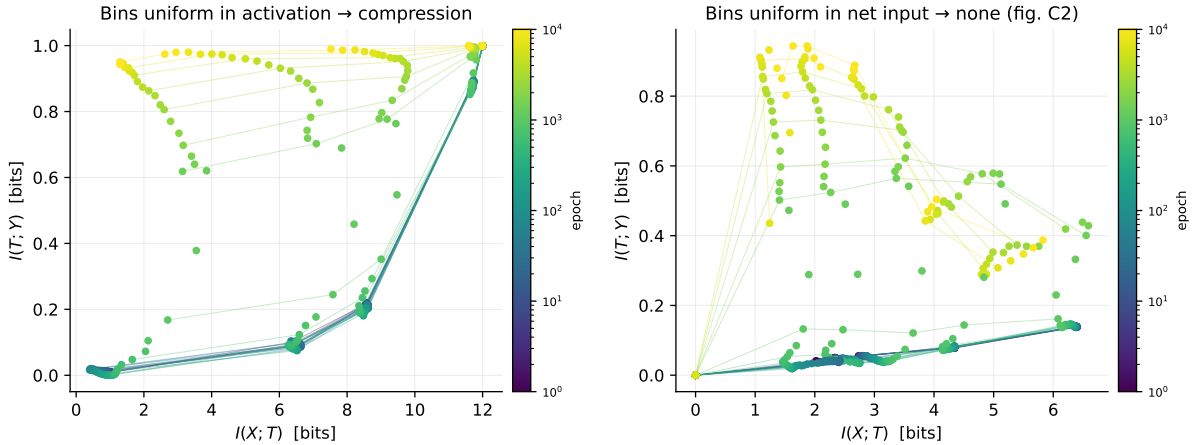


Figure 3: The same network, the same weights, the same training run — measured with two different sets of bin edges. Compression appears or disappears according to the analyst's choice.

The Kraskov estimator is invalid for ReLU. This is a finding beyond the paper. Applying (9) to our ReLU layers returns entropies as low as -88 bits, far below the $\log_2 4096 = 12$ bit ceiling any genuine entropy here must respect. The cause: a ReLU unit that is off outputs *exactly* zero, so 44.6 % of ReLU activation vectors are bit-identical (versus 0 % for `tanh`). The distribution has an *atom* at zero, making it mixed discrete–continuous, for which differential entropy is not defined; Kozachenko–Leonenko assumes absolute continuity. Operationally, when $r_i = 0$ the guard ε takes over and each duplicated point contributes $(d/P)\log\varepsilon \approx -36.8\, d/P$ nats, so the estimate reports *how many points coincide*. The paper introduces $\varepsilon = 10^{-16}$ to "prevent infinite terms" without noting that the estimate is then meaningless. We therefore read the `tanh` panel — which agrees with the other estimators, layers 4 and 5 losing 8.5 and 21.3 bits — and discard the ReLU panel.

5 Exact information in deep linear networks

The results so far could in principle be blamed on estimation. Deep linear networks remove that possibility: every hidden layer is jointly Gaussian with input and output, so (10)–(11) are exact. They also answer a second objection — that compression might come from neurons becoming

activation	double-saturating?	max compression (bits)
`tanh`	yes	0.69
`softsign`	yes	0.11
`relu`	no	0.10
`softplus`	no	0.01

Table 3: KDE estimator, mean of five seeds. Compression tracks saturation behaviour.

correlated or from irrelevant directions being projected out, neither visible in a one-neuron model.

No compression occurs in any layer, at any depth, under SGD or BGD, over 500 epochs or 20 000. The largest single-step decrease anywhere is $\sim 10^{-5}$ bits, four orders of magnitude below the overall rise, and no layer ever ends below its peak.

5.1 The dissociation

Table 4 collects every configuration. If Claim 2 held, the "compresses" and "generalises" columns would agree. All four combinations occur.

network	compression	compresses?	gap / E_g ratio	generalises?
`tanh`, 80 % data	10.66 bits	yes	0.019	yes
ReLU, 80 % data	0.47 bits	no	0.014	yes
`tanh`, 30 % data	7.99 bits	yes	0.063	no
ReLU, 30 % data	0.62 bits	no	0.055	no
linear, BGD (fig. 3)	0.00 bits	no	1.6×	yes
linear, SGD b=5 (fig. 4)	0.00 bits	no	20.3×	no

Table 4: Compression and generalisation vary independently. Neither implies the other, refuting Claim 2.

5.2 Identical functions, different information

The sharpest illustration that the measured quantity is not a property of the function computed. Take $\hat{y} = w_2 w_1 x$ and rescale $\tilde{w}_1 = w_1/c$, $\tilde{w}_2 = c\, w_2$. Every member of this family computes *exactly the same map* and so generalises identically. But

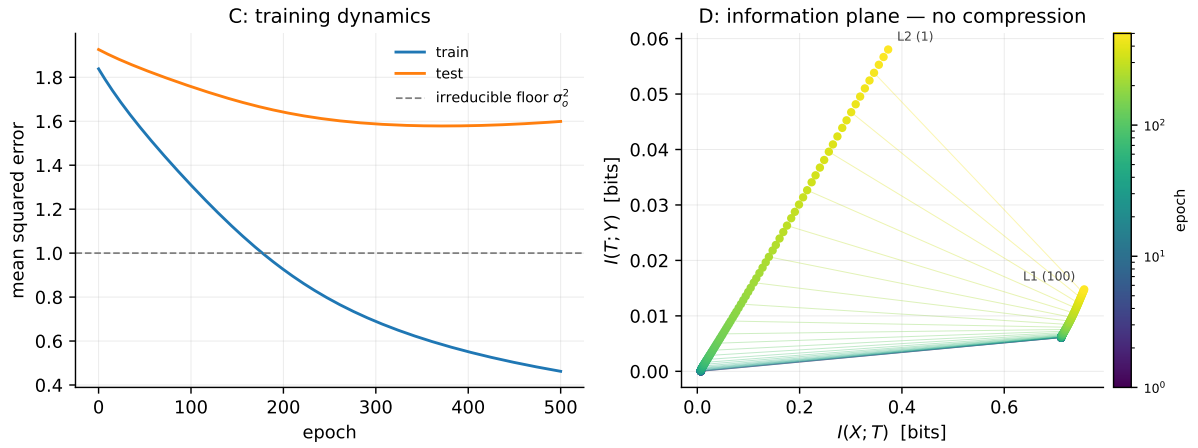
$$\mathrm{I}(T;X) = \tfrac{1}{2}\log\left(w_1^2/c^2 + \sigma_{MI}^2\right) - \tfrac{1}{2}\log\sigma_{MI}^2, \tag{17}$$

which depends on c: from 4.32 bits at $c = 0.1$ down to 0.03 bits at $c = 10$, for an unchanged input–output map. Mutual information is invariant under invertible reparametrisation; $T = h + Z$ is not, because the noise enters at a fixed scale.

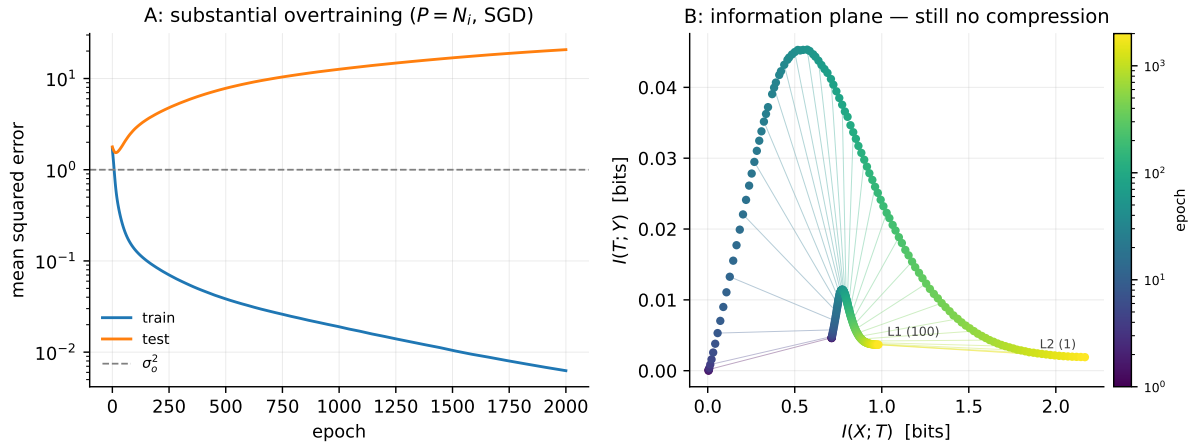
6 Compression is not caused by SGD noise

Claim 3 states that compression arises from the diffusion phase of SGD. It fails on two independent grounds.

Theoretically. The argument runs: diffusion drives the weight distribution to maximum entropy subject to a training-error constraint; maximum entropy maximises $\mathrm{H}(X \mid T)$; hence $\mathrm{I}(X;T) = \mathrm{H}(X) - \mathrm{H}(X \mid T)$ is minimised. The middle step does not follow. The maximum-entropy distribution is over *weights across training runs*; $\mathrm{H}(X \mid T)$ is uncertainty about *inputs*



(a) Batch gradient descent, 500 epochs: generalises well.



(b) SGD, 5 samples per batch: overfits by a factor of 13.5.

Figure 4: Exact information planes in deep linear networks. The two settings have identical architecture and data-generating process and *indistinguishable* information planes — both monotonically increasing, neither compressing — yet their generalisation differs by more than an order of magnitude.

drawn from the data distribution, given one fixed network. These are different random variables, and there is no general reason a sample from the former maximises the latter.

Empirically. The claim makes a testable prediction: remove the randomness and compression should vanish. Full-batch gradient descent has no sampling noise and no diffusion. Table 2 shows `tanh` compression under BGD is 17.9 bits — *larger* than the 10.7 bits under SGD. The prediction fails in the wrong direction.

The SNR transition is real but unrelated. Both networks lose one to two orders of magnitude of gradient SNR over training (20 to 68× per layer), with no systematic difference between the compressing and non-compressing network. The transition is a property of *arriving at a minimum*: the mean gradient vanishes there by definition, while per-sample disagreement does not. It was described long before the IB literature as the transient/stochastic phases [Murata, 1998].

The decisive case is a 1–1–1 linear network with small initial weights (Figure 5). With a single hidden unit whose weight only grows, compression is impossible *by construction* — yet

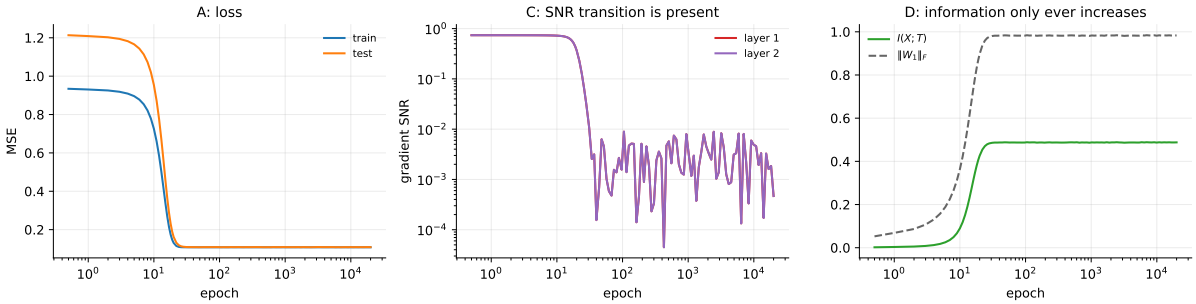


Figure 5: A 1–1–1 linear network. The gradient SNR undergoes a textbook drift→diffusion collapse (16 502×) while the weight norm *grows* and $\mathrm{I}(X;T)$ only ever increases. The SNR transition is therefore independent of compression.

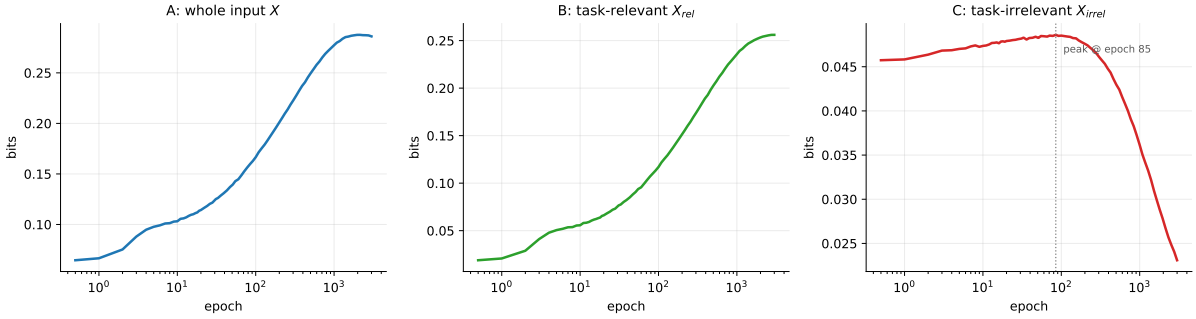


Figure 6: **A:** information about the whole input does *not* compress. **B:** information about the relevant subspace rises robustly. **C:** information about the irrelevant subspace rises and then falls — genuine compression, invisible in panel A.

the SNR falls by a factor of 16 502.

7 When compression does occur — and when

One intuition survives all of the above: if some input channels carry only noise, good generalisation *requires* discarding them. The paper takes this seriously, partitioning the input into a task-relevant block and a task-irrelevant block with the teacher's weights on the latter set to exactly zero. Using (12), everything is again exact.

So the intuition is half right. $\mathrm{I}(X_{\mathrm{irrel}};T)$ peaks at 0.049 bits and falls to 0.023 bits, a 53 % reduction, while $\mathrm{I}(X;T)$ rises throughout. The sub-additivity identity (13) explains the apparent paradox: the synergy term $\mathrm{I}(X_{\mathrm{rel}};X_{\mathrm{irrel}}\mid T)\geq 0$ absorbs the difference. We verify numerically that this term is non-negative throughout.

The timing. The IB claim is not merely that compression happens but that it occupies a *separate second phase*. The paper asserts the two are concurrent; we quantify it. Compression of the irrelevant subspace begins at epoch 85, at which point fitting of the relevant subspace is only 39 % complete (33 % and 7 % for two other seeds). Fitting reaches 90 % completion only at epoch 995, more than ten times later. **The two processes are interleaved, not sequential.**

Consequence for methodology. The compression that a task actually demands is invisible in $\mathrm{I}(X;T)$, the quantity the standard information plane plots. Detecting it requires a subspace-resolved measure. This is a positive finding: the information-theoretic framing is not useless, but the aggregate statistic is the wrong instrument.

8 Extension: does a transformer compress?

The paper's mechanism is stated for the activation function, and modern architectures use non-saturating ones. But a transformer block contains two operations the paper never examined, both *bounded*:

- **softmax attention** [Vaswani et al., 2017]: each output is a convex combination of value vectors, confined to their convex hull, and the attention weights themselves saturate towards one-hot as logits grow;
- **LayerNorm** [Ba et al., 2016]: projects the residual stream onto a sphere of fixed radius $\sqrt{d}$, so however large the weights grow the representation cannot spread.

Both saturate in the relevant sense. The paper's own mechanism therefore predicts that a transformer should exhibit apparent compression driven by its *normalisation and attention* rather than its feed-forward nonlinearity, and that removing LayerNorm should remove it.

We test this on the *same* 12-bit input space, fed as 12 tokens, so that the 4096 patterns remain the entire input distribution and binning-based mutual information stays exact and directly comparable with §4. The model is a two-block encoder, $d_{\text{model}} = 8$, two heads, $d_{\text{ff}} = 16$, with a narrowing head $8 \to 4 \to 2$ mirroring the Tishby architecture — 1310 parameters in total. All sequence-valued representations are mean-pooled over positions; a full $12 \times d$ representation would be high-dimensional enough that every input received its own bin, pinning $\mathrm{I}(X;T)$ at 12 bits with no dynamics (the failure mode of §4.3). We cross the feed-forward nonlinearity (GELU, ReLU, tanh) with LayerNorm on and off.

8.1 Result: the normalisation axis dominates

feed-forward	with LayerNorm	without LayerNorm	effect of LN
GELU	2.47 bits	0.01 bits	+2.45
ReLU	3.77 bits	0.07 bits	+3.70
tanh	3.68 bits	2.15 bits	+1.53
mean effect of adding LayerNorm			**+2.56**
spread across activations (with LN)			1.31
spread across activations (without LN)			2.14

Table 5: Total compression in the tiny transformer. Adding LayerNorm moves the number by more than swapping the nonlinearity does — the reverse of the paper's finding for MLPs, and what its own mechanism predicts. All six configurations reach 100 % train accuracy and 96.6 % to 98.7 % test.

The hypothesis is supported. The sharpest comparison is GELU: a purely non-saturating architecture shows *essentially no compression* (0.01 bits) until a normalisation layer is added, whereupon it compresses 2.47 bits. Note also that tanh is the only activation still compressing appreciably without LayerNorm (2.15 bits) — the paper's own mechanism showing through the transformer.

Two results cut against the naive reading, and both are informative.

Compression appears in the feed-forward activations, not in attention or the residual stream. With LayerNorm, the feed-forward representations account for 0.34 bits to 1.89 bits against 0.15 bits to 0.24 bits for attention, residual and pooled representations combined. LayerNorm *causes* the effect but is not *where* it appears: sitting on the input to the FFN, it caps the scale of what the FFN sees, so the FFN's hidden activations cannot spread as the weights grow. The cap is applied in one place and measured in another.

Removing LayerNorm makes attention saturate far harder — and compress far less. Without normalisation, attention entropy collapses from 3.56 to 0.43 bits, nearly one-hot; with it, only 3.50 → 3.00 bits. Yet the un-normalised runs barely compress. The other diagnostic resolves this: without LayerNorm the residual-stream norm explodes from 2.1 to ≈ 100, while with it the norm stays near 9 to 13. Unbounded spreading raises entropy far faster than sharpened attention lowers it.

A sharpened statement of the mechanism. That second point refines the paper's thesis. What produces apparent compression is not "saturation" loosely construed — it is a representation whose **scale stops growing while the weights keep growing**. A tanh unit has that property through its asymptotes; LayerNorm has it by construction; softmax attention, on its own, does *not*, because the value vectors it averages can still grow without bound. Convexity bounds the output relative to its inputs, which is not the same as bounding it absolutely.

Consequence. Modern architectures are routinely described as "ReLU-family" and therefore, on a naive reading of the paper, exempt from the compression story. LayerNorm is present in essentially every transformer, and it imposes exactly the fixed ceiling that drives the effect. Any information-plane analysis of a normalised architecture must account for the normalisation before attributing a trajectory to learning.

Caveats. One seed per configuration, one architecture, one small synthetic task; this is an exploratory extension rather than a systematic study. Pooling over positions discards information, though the LayerNorm/activation contrast is measured under identical pooling. And the binning range is set per run from observed extremes, so the normalised and un-normalised runs are binned over different ranges — the same arbitrariness this whole report is about, now applying to our own extension.

9 Conclusions

1. **Claim 1 (two phases) is false in general.** The compression phase appears for double-saturating nonlinearities and not for single-sided or unbounded ones, under three independent estimators and in an exactly solvable model. Changing one identifier from `tanh` to `relu` removes it while improving test accuracy.
2. **Claim 2 (causality) is false.** All four combinations of (compresses, generalises) are realisable. In deep linear networks, two configurations with identical information planes differ in generalisation error by a factor of twenty.
3. **Claim 3 (SGD diffusion) is false.** Deterministic full-batch descent compresses *more* than SGD. The gradient-SNR transition occurs identically in networks that do not compress, and in a network where compression is architecturally impossible.
4. **The deeper issue is definitional.** For a deterministic continuous network $\mathrm{I}(h; X) = \infty$. Every finite information-plane number is a joint property of the network *and* an arbitrary binning or noise assumption. Holding the network fixed and changing only the bin edges moves the deepest layer's compression from 5.59 to 0.32 bits; rescaling weights without changing the function computed moves $\mathrm{I}(T; X)$ over two orders of magnitude.
5. **But compression is not an empty notion.** When part of the input is genuinely irrelevant, the network does discard it — by 53 % here — concurrently with fitting rather than in a second phase, and invisibly to $\mathrm{I}(X; T)$. The information-theoretic framing survives; the aggregate statistic and the two-phase narrative do not.

What we would add to the paper. Three things: the quantified timing of §7, which converts an assertion into a measurement; the Kraskov failure of §4.4, which affects any future use of that estimator on ReLU networks; and the observation that architectures built entirely from non-saturating nonlinearities may still compress artefactually through normalisation and attention.

References

Madhu S Advani and Andrew M Saxe. High-dimensional dynamics of generalization error in neural networks. *arXiv preprint arXiv:1710.03667*, 2017.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Gal Chechik, Amir Globerson, Naftali Tishby, and Yair Weiss. Information bottleneck for Gaussian variables. *Journal of Machine Learning Research*, 6:165–188, 2005.

Thomas M Cover and Joy A Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

Artemy Kolchinsky and Brendan D Tracey. Estimating mixture entropy with pairwise distances. *Entropy*, 19(7):361, 2017.

LF Kozachenko and Nikolai N Leonenko. Sample estimate of the entropy of a random vector. *Problemy Peredachi Informatsii*, 23(2):9–16, 1987.

Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. Estimating mutual information. *Physical Review E*, 69(6):066138, 2004.

Simon Laughlin. A simple coding procedure enhances a neuron's information capacity. *Zeitschrift für Naturforschung C*, 36(9-10):910–912, 1981.

Noboru Murata. A statistical study of on-line learning. In *On-line Learning in Neural Networks*, pages 63–92. Cambridge University Press, 1998.

Andrew M Saxe, Yamini Bansal, Joel Dapello, Madhu Advani, Artemy Kolchinsky, Brendan D Tracey, and David D Cox. On the information bottleneck theory of deep learning. *Journal of Statistical Mechanics: Theory and Experiment*, 2019(12):124020, 2019. doi: 10.1088/1742-5468/ab3985. Updated version of the ICLR 2018 paper of the same title.

Ravid Shwartz-Ziv and Naftali Tishby. Opening the black box of deep neural networks via information. In *arXiv preprint arXiv:1703.00810*, 2017.

Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle. In *IEEE Information Theory Workshop (ITW)*, pages 1–5, 2015.

Naftali Tishby, Fernando C Pereira, and William Bialek. The information bottleneck method. In *Proceedings of the 37th Allerton Conference on Communication, Control and Computing*, pages 368–377, 1999.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

A Software

The accompanying library `ibdl/` contains: `data.py` (datasets), `models.py` (MLPs, deep linear networks), `minimal.py` (the exactly solvable model), `estimators/` (the four estimators), `train.py` and `linear_np.py` (instrumented trainers), `transformer.py`, `experiments.py`, `planes.py`, `cache.py` and `plotting.py`. Validation lives in `tests/`; all 46 checks pass. Notebooks `01`–`07` reproduce every figure.